A Real-Time Graphical Representation of Various Path Finding Algorithms for Route Optimisation

Ravali Attivilli, Afraa Noureen, Vaibhavi Sachin Rao, and Siddhaling Urolagin

Abstract A country's economy relies heavily on transport—it connects us to the rest of the world and facilitates growth. There are many types of navigation systems for cars, including Google Maps, GPS, etc. These systems give the best route possible but are also susceptible to errors. Numerous algorithms have been developed and compared by researchers to determine the optimal route planning approach. A comparative analysis of various algorithms is included in this review, including Dijkstra, *A**, uniform cost search, and Bellman-Ford (dynamic programming). These algorithms are compared to determine which one is most suitable for finding the optimal routes for the given dataset.

Keywords Route planning · *A** algorithm · Dijkstra algorithm · Uniform cost search algorithm · Dynamic programming

1 Introduction

According to Harvard University research done in 2019, people living in Los Angeles spend an average of 119 h a year stuck in traffic as the number of vehicles increases**.** We must incorporate technology to increase productivity and solve this problem. A transformation of the transportation industry is possible using the intersection of the digital and the physical realms. As a result, we are faced with the shortest path problem. This problem occurs quite often in graph theory and affects many aspects of our daily lives. Route planning and determining the shortest route are the

R. Attivilli (✉) · A. Noureen · V. S. Rao · S. Urolagin
BITS Pilani, Dubai, UAE
e-mail: f20190224@dubai.bits-pilani.ac.in

A. Noureen
e-mail: f20190239@dubai.bits-pilani.ac.in

V. S. Rao
e-mail: f20190151@dubai.bits-pilani.ac.in

S. Urolagin
e-mail: siddhaling@dubai.bits-pilani.ac.in

S. Das et al. (eds.), *Advances in Data-driven Computing and Intelligent Systems*, Lecture Notes in Networks and Systems 653, https://doi.org/10.1007/978-981-99-0981-0_37

primary aspects of this research. And the distance between the target destination and the starting point is minimised in the process. This problem is addressed by using algorithms that determine the shortest route. Time complexity, memory usage, search space, and accuracy are a few of the metrics used to evaluate algorithms that find the shortest route. These algorithms are used in GPS and Google Maps to determine the shortest route, which also considers external factors such as tolls, traffic, transportation modes, and adding extra stops.

In this paper, we aim to provide a practical understanding of shortest path algorithms so that relevant parties will be able to design applications that will assist their target audience in finding the shortest path. We utilise the concept of graphs to provide a visual representation of the area around Dubai International Academic City (DIAC) and then compare the paths that are generated using the Dijkstra algorithm, *A** algorithm, uniform cost search method, and Bellman-Ford algorithm.

2 Related Works

This section discusses the previous work proposed by various authors on finding shortest path and/or route optimisation. According to [1], the authors implemented shortest path algorithms like Bellman-Ford and Dijkstra's algorithm on roads of Bandung city to help users in navigation. They employed a five-step methodology: research, verification, comparison unit, adjustment analysis, and reconciliation. According to their results, the number of nodes was reduced and the negative weights could be handled using Bellman-Ford algorithm but not with Dijkstra's algorithm. Haria et al. [2] understood the working of Google Maps and how it provides the shortest route. They performed the analysis on real-time reports collected from smart devices and concluded that both Dijkstra's and *A** were highly effective than Bellman-Ford algorithm. In [3], Alzubaidi and Al-Thani modified Dijkstra's algorithm to identify a route in terms of road safety and quality. The proposed approach resulted in low accuracy as the dataset consisted of most popular cities in Yemen and data retrieval difficulties due to current situation in Yemen. The study involved the use of Dijkstra's, Bellman-Ford and Floyd Warshall algorithms. AbuSalim et al. [4] proposes a paper on research studies where Dijkstra's and Bellman-Ford algorithms are compared based on complexity and performance in terms of shortest path optimisation. While Dijkstra's is beneficial for large numbers of nodes, it takes up a great deal of storage space and has a high time complexity. In another paper [5], a comparison between Floyd Warshall and greedy algorithm was drawn to find out the optimal path. It was found that Floyd Warshall gave accurate results but took more time than greedy algorithm.

3 Dataset

For the purpose of this experiment, we took the area around Dubai International Academic City and Dubai Silicon Oasis, UAE and identified 48 important locations. Each location is considered as a node in the graph, and the connection between them is represented by an edge between the nodes. The latitude and longitude associated with each location are recorded as x–y coordinates and stored in our dataset in the form of a data dictionary so that they can be represented in the form of a graph. Figure 1 represents the actual GPS coordinates of the 48 locations that we have identified, in a real-time Google Maps simulator that was created for the sole purpose of this study.

Fig. 1 Real-time map with all connections of the 48 locations represented using our Google Maps simulator

4 Methodology

In order to accomplish the objectives of our research, the study was divided into two different phases. In the first phase, we choose a starting point and a destination to compute the shortest path between them, the time taken to achieve the desired result was also noted. In the second phase, we simulate real-time traffic scenarios and make some roads in the path unavailable for use. We then run the simulation again to find the new paths taken and record the time. This trial is repeated for all the algorithms that were being studied.

Dijkstra's Algorithm—Greedy Method

Algorithm 1 Dijkstra's Algorithm

```
1: function dij(A,Q)
2:     for each vertex X in A
3:         dist[X] = infinite
4:         prev[X] = NULL
5:         if X != Q then add X to Priority Queue Y
6:     dist[Q] = 0
7:     while Y IS NOT EMPTY do
8:         P = Extract MIN from Y
9:         for each unvisited neighbour X of P
10:            tempDist = dist[P] + edge_wt(P,X)
11:            if tempDist < dist[X] then
12:                dist[X] = tempDist
13:                prev[X] = P
14:    end while
15:    return dist[ ], prev[ ]
```

Depending on how a graph is represented, Dijkstra's algorithm can be used to find shortest distances or the cheapest course of action. Essentially, find the shortest leg each time, starting at the end and working backwards.

*A** Algorithm

Algorithm 2 A* Algorithm

```
1: initialize
2:      opn_list = {initial}
3:      closd_list = {}
4:      g(initial) = 0
5:      h(initial) = heuristic_func(initial,final)
6:      f(initial) = g(initial) + h(initial)
7: while opn_list is not empty do
8:      x = Node on top of opn_list, with least f
9:      if x == final
10:             return
11:     remove x from opn_list
12:     add x to closd_list
13:     for each y in child(x)
14:             if y in closd_list then
15:                     continue
16:             cost = g(x) + dist(x,y)
17:             if y in closd_list and cost < g(y) then
18:                     remove y from opn_list as new path is better
19:             if y is closd_list and cost < g(y) then
20:                     remove y from closd_list
21:             if y is not opn_list and y is not closd_list then
22:                     add y to opn_list
23:                     g(y) = cost
24:                     h(y) = heuristic_func(y,final)
25:                     f(y) = g(y) + h(y)
26: return failure
```

An *A** begins by first calculating the cost of travelling to neighbouring nodes, then choosing the node with the lowest cost. *A**'s efficiency depends on the heuristic value.

The calculation of the value can be done as follows:

$$f(y) = g(y) + h(y). \tag{1}$$

$g(y)$ Actual cost from initial node to y.
$h(y)$ Eestimation cost from y to goal node.

Uniform Cost Search Method

Algorithm 3 Uniform Cost Search Method

```
1: function UCS (Graph, start, target)
2:        Add the starting node to the open list. The node has zero distance value from itself
3:        while True do
4:                if open is empty then
5:                  break
6:                selectedNode = remove from open list, the node with min distance value
7:                  if selectedNode == target then
8:                          calculate path
9:                          return path
10:               add selectedNode to closed list
11:               newNodes = get the children of the selectedNode
12:               if the selected node has children then
13:                       for each child in children
14:                       calculate the distance value of child
15:                       if child not in closed and open lists then
16:                            child.parent = selectedNode
17:                            add the child to open list
18:                       else if child in open list then
19:                            if the distance value of the child is lower than the corresponding node in
open list then
20:                                      child.parent = selectedNode
21:                                      add the child to open list
```

This algorithm traverses a weighted graph or tree. When the costs for every edge are different, this algorithm comes into play. Its primary objective is to establish a route to the goal node that has the lowest cumulative cost. Nodes are expanded based on the cost of the path distance from the root node. A graph or tree whose optimal cost needs to be determined can be solved using this method. Under UCS, the lowest cumulative cost is given maximum priority using the priority queue. When all the edges have the same path cost, UCS is equivalent to BFS.

Bellman-Ford—Dynamic Programming

Algorithm 4 Bellman Ford Algorithm

```
1: function algo_bellman(A,S)
2:        for each vertex X in A
3:                dist[X] = infinite
4:                        prev[X] = NULL
5:        dist[S] = 0
6:        for each vertex X in A
7:                for each edge (Y,X) in A
8:                        tempDist = dist[Y] + edge_wt(Y,X)
9:                        if tempDist < dist[X] then
10:                               dist[X] = tempDist[X]
11:                               prev[X] = Y
12:       for each edge (Y,X) in A
13:               if dist[Y] + edge_wt(Y,X) < dist[X]
14:                       print 'Negative Cycle Exists'
15:       return dist[], prev[]
```

A dynamic programming solution, for example, breaks down optimisation problems into smaller pieces and reuses previous calculations to be more efficient than a recursive approach. The algorithm approximates function $j()$ which gives the shortest path from initial to final node.

The Bellman-Ford equation is written as

$$j(\text{node } A) = \min[C(\text{node } A, \text{node } B) + j(\text{node } B)] \text{ over all node } B \times \text{which are neighbours of node } A, \quad (2)$$

where $C()$ = transition cost and $j()$ = cost from initial to final node.

5 Results and Discussion

Our experiment was divided into two phases. In each phase, we took a total of three test cases (paths) and recorded the route it takes to reach the destination. We then tested each path with four algorithms, i.e. Dijkstra's, A^*, uniform cost search, and Bellman-Ford, and the time taken to execute each algorithm was recorded. For the first phase, we assumed that there was no traffic and recorded the results. In the second phase, we simulated traffic and found out the time taken to find an alternative route and the new traversing path. All the obtained results were then plotted into the map.

5.1 Finding Shortest Path from Set El Sham (Node 0) to McDonald's (Node 46)

For Table 1, we chose the starting node as Set El Sham (0) and the destination node as McDonald's (0). The shortest route calculated in phase 1 is [0, 7, 46], and the shortest route in phase 2 (*paths 0–7 and 7–46 are unavailable*) is [0, 19, 7, 45, 46] (Figs. 2 and 3).

As seen from the results in phase 1 and phase 2, the execution time of each algorithm arranged in increasing order is

$$A^* \ll \text{Dijkstra's} \ll \text{Uniform cost search} \ll \text{Bellman-Ford}.$$

Table 1 Experimental results for the route from Set El Sham (0) to McDonald's (46)

	Without traffic management (phase 1)				With traffic management (phase 2)			
	Dijkstra's	*A**	Uniform cost search	Bellman-Ford	Dijkstra's	*A**	Uniform cost search	Bellman-Ford
Traversing path	[0, 7, 46]	[0, 7, 46]	i. [0, 7, 46] ii. [0, 7, 45, 46]	[0, 7, 46]	[0, 19, 7, 45, 46]	[0, 19, 7, 45, 46]	[0, 19, 7, 45, 46]	[0, 19, 7, 45, 46]
Time taken to execute (in sec)	1.0057	1.0056	1.0133	1.0439	1.00680	1.00434	1.01254	1.03711

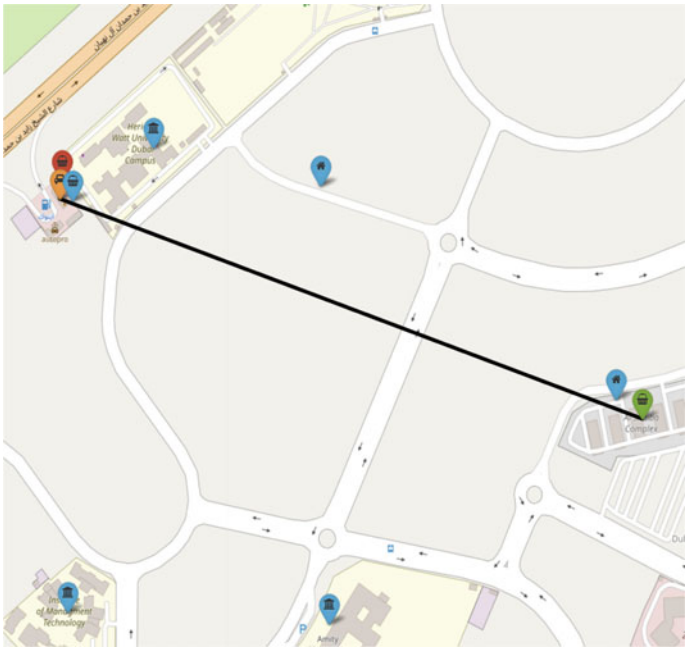

Fig. 2 Graphical representation of the route [0, 7, 46] (without traffic simulation)

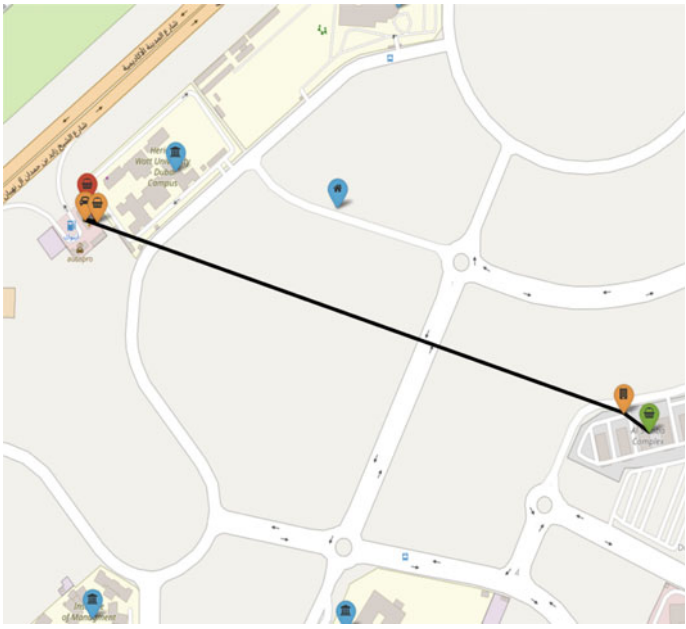

Fig. 3 Graphical representation of the route [0, 19, 7, 45, 46] (with traffic simulation)

5.2 *Finding Shortest Path from IMT Business School Dubai (Node 9) to German International School Dubai (Node 29)*

For Table 2, we chose the starting node as IMT Business School Dubai (9) and the destination node as German International School Dubai (29). In phase 1, the shortest route calculated is [9, 27, 29] (refer to Fig. 4). As seen from the experimental results in this phase, the execution time of each algorithm arranged in increasing order is

$$\text{Dijkstra's} \ll A^* \ll \text{Uniform cost search} \ll \text{Bellman-Ford}.$$

In phase 2, the shortest route calculated is [9, 10, 27, 28, 29] (refer to Fig. 5). Similarly, arranging the observed execution times for each algorithm in this phase in increasing order produces the same result.

5.3 *Finding Shortest Path from BITS-Dubai (Node 1) to Golden Pak Restaurant FZ LLC (Node 44)*

For Table 3, we chose the starting node as BITS-Dubai (1) and the destination node as Golden Pak Restaurant FZ LLC (44). In phase 1, the shortest route calculated is [1, 19, 44] (refer to Fig. 6). As seen from the experimental results in this phase, the execution time of each algorithm arranged in increasing order is

$$A^* \ll \text{Dijkstra's} \ll \text{Uniform cost search} \ll \text{Bellman-Ford}.$$

In phase 2, the shortest route calculated is [1, 0, 44] (refer to Fig. 7). Similarly, arranging the observed execution times for each algorithm in this phase in increasing order produces the same result.

Table 2 Experimental results for the route from IMT Business School Dubai (9) to German International School Dubai (29)

	Without traffic management (phase 1)				With traffic management (phase 2)			
	Dijkstra's	*A**	Uniform cost search	Bellman-Ford	Dijkstra's	*A**	Uniform cost search	Bellman-Ford
Traversing path	[9, 27, 29]	[9, 27, 29]	i. [9, 27, 29] ii. [9, 27, 28, 29]	[9, 27, 29]	[9, 10, 27, 28, 29]	[9, 10, 27, 28, 29]	[9, 10, 27, 28, 29]	[9, 10, 27, 28, 29]
Time taken to execute (in sec)	1.00547	1.00691	1.01035	1.10689	1.00823	1.00534	1.00899	1.11679

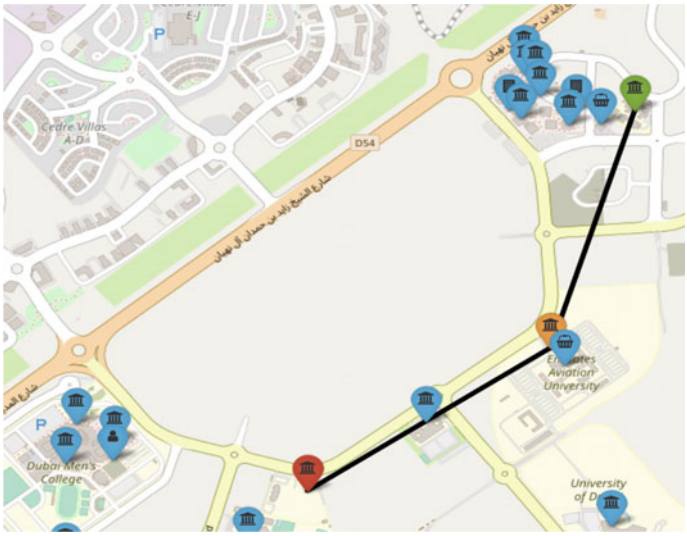

Fig. 4 Graphical representation of the route [9, 27, 29] (without traffic simulation)

Fig. 5 Graphical representation of the route [9, 10, 27, 28, 29] (with traffic simulation)

Table 3 Experimental results for the route from BITS-Dubai (1) to Golden Pak Restaurant FZ LLC (44)

	Without traffic management (phase 1)				With traffic management (phase 2)			
	Dijkstra's	*A**	Uniform cost search	Bellman-Ford	Dijkstra's	*A**	Uniform cost search	Bellman-Ford
Traversing paths produced	[1, 19, 44]	[1, 19, 44]	i. [1, 19, 44] ii. [1, 0, 44] iii. [1, 19, 20, 44] iv. [1, 19, 20, 24, 44] v. [1, 19, 20, 25, 44]	[1, 19, 44]	[1, 0, 44]	[1, 0, 44]	i. [1, 0, 44] ii. [1, 0, 20, 44] iii. [1, 0, 20, 24, 44] iv. [1, 0, 20, 25, 44]	[1, 0, 44]
Time taken to execute (in sec)	1.00711	1.00275	1.01324	1.04941	1.00561	1.00095	1.01173	1.05492

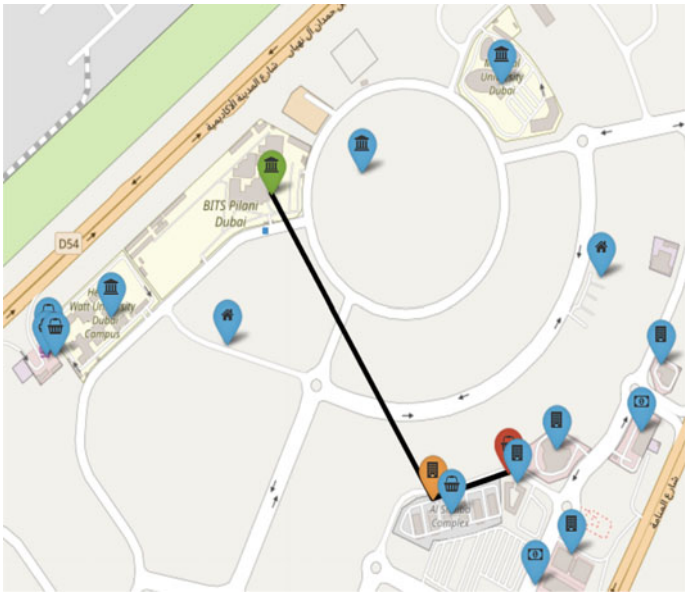

Fig. 6 Graphical representation of the route [1, 19, 44] (without traffic simulation)

Fig. 7 Graphical representation of the route [1, 0, 44] (with traffic simulation)

6 Conclusion and Future Scope

This paper reviews multiple shortest path algorithms. Each of these algorithms was tested to find the optimal path with and without the simulation of traffic. While all the algorithms produced the same route for all the test cases, their execution time varied greatly. In Sect. 5.1, we observe that *A** performs the best (with execution times 1.0056 and 1.00434 s). Similarly in Sect. 5.2, *A** performs the best (with execution times 1.00691 and 1.00534 s). Finally in Sect. 5.3, we observe *A** outperforming the other algorithms (with execution times 1.00275 and 1.00095 s). Once the route was obtained, the results were plotted on the map—where the start node was marked green, the nodes traversed through were marked orange and the destination node was marked red. This was done to ease the comprehension of the results obtained and to view them in a real-life scenario. Thus, we were able to conclude that the *A** algorithm produces the most optimal results in the shortest time possible, followed by the greedy algorithm (Dijkstra's), then the uniform cost search method and lastly the Bellman-Ford algorithm.

The future work of this study will focus on obtaining the results in real time and on a larger dataset for deeper analysis.

References

1. Pramudita R, Heryanto H, Trias Handayanto R, Setiyadi D, Arifin R, Safitri N (2019) Shortest path calculation algorithms for geographic information systems. In: 2019 fourth international conference on informatics and computing (ICIC)
2. Haria V, Shah Y, Gangwar V, Chandwaney V, Jain T, Dedhia Y (2019) The working of google maps, and the commercial usage of navigation systems. IJIRT 6
3. Alzubaidi M, Al-Thani D (2021) Finding the safest path: the case of Yemen. In: 2021 international conference on information technology (ICIT)
4. Abu Salim S, Ibrahim R, Zainuri Saringat M, Jamel S, Abdul Wahab J (2020) Comparative analysis between Dijkstra and Bellman-Ford algorithms in shortest path optimization. IOP Conf Ser Mater Sci Eng 917:012077
5. Azis H, Mallongi R, Lantara D, Salim Y (2018) Comparison of Floyd-Warshall algorithm and greedy algorithm in determining the shortest route. In: 2018 2nd East Indonesia conference on computer and information technology (EIConCIT)